# Music format – Pilotwings 64 (USA)

Findings behind the Music tab of Hawk's Hangar. The game uses the N64 SDK audio library (libaudio), not the EAD driver of Super Mario 64, so there are no .m64 files.

## Audio block (behind the file system, addresses hardcoded in the audio code)

| Part | ROM offset | Size | Content |
|---|---|---|---|
| Sequences (ALSeqFile "S1") | 0x618B70 | 84,208 | 31 songs, compact MIDI |
| Instrument bank (.ctl, "B1") | 0x62D460 | 16,496 | 1 bank, 100 places, 48 instruments, 127 sounds |
| Samples (.tbl) | 0x6314D0 | 833,296 | VADPCM, 22,050 Hz |

S1 file: "S1", u16 count, then (u32 offset, u32 length) per song; songs 4-byte aligned, file padded to 16. The game copies one song at a time into a buffer as large as the longest song.

## Compact MIDI (ALCSeq, alCSPlayer)

- Header: 16 × u32 track offsets (0 = unused) + u32 division (768 in all songs).
- Track 0 of every song only holds tempo; each other track plays exactly one MIDI channel.
- Note-on is followed by the note length (varlen); there are no note-off events.
- `FE hi lo n`: repeat n earlier bytes starting (hi·256+lo) bytes before the code; may point into other tracks. `FE FE` = literal 0xFE. Repeated bytes are read raw.
- Loops: `FF 2E xx xx` (start), `FF 2D` + 6 raw bytes (count, current count, u32 distance back).
- Channel volume: a program change sets the channel volume to the instrument's own volume (`__setInstChanState`, csplayer.c). The songs send CC 7 before their program change at tick 0, so a CC 7 at the start is overwritten right away (found 2026-10-14 with the channel states in a savestate). A note with velocity 0 is treated as a note off (`if (vel != 0)` in csplayer.c).

## Instruments

- Program numbers index the bank directly (not General MIDI). Empty places crash the game when a note is played, so the editor only offers the 48 used ones.
- Channel 10 (index 9) starts with the bank's percussion instrument = program 1 (Drums 1); all other channels start with the first instrument (program 0). Songs 11 and 20 play drums on channel 10 without a program change.
- Instrument names and categories were given by the project owner after listening (2026-10-09).

## Editor behavior

- Instrument change = program numbers changed in place.

- Mute = the velocity of every note of the channel set to 0, in place (since 2026-10-14; before, the CC 7 values were set to 0, which had no effect because of the program changes). Checked in the emulator: with all voices of all songs muted, the title screen only plays its sound effects.
- A byte shared by repetitions with different needs is written out (the repetition expanded); missing program changes are inserted at the track's first event, and all track offsets, repetition distances and loop distances across the change are corrected.
- Every result is re-parsed and compared event by event with the expected result.
- Changes are stored per song in project.json (`Music`); the build writes a new S1 file. If it grows, the bank and samples move behind it and the code references are patched.

### Loops in the original songs

- All looping songs loop as a whole: every track has `FF 2E 00 FF` (loop start) and `FF 2D FF FF <u32>` (loop end, count 255 = forever) at the same ticks. The jump goes to the time value right behind the loop start event. Jingles (landing, crash, …) have no loop.

### MIDI export and import

- Export: MIDI type 1, track 1 = tempo + markers `loopStart` / `loopEnd`, one track per channel (named after voice and instrument). Program numbers are the game's instruments. The current project version is exported (imported song + instrument changes + mutes).
- Import: type 0 and 1, ticks per quarter note. Loop from markers `loopStart` / `loopEnd` (also "loop start", "[", "]") or CC 102/103. Notes get their length from the note-off; notes behind the loop end are dropped; unknown programs fall back to the channel's default instrument (with a warning); sysex is dropped. One track per channel, tempo in the first.
- Packing: running status plus repeated runs (`FE hi lo n`, at least 16 bytes, sources only plain bytes, never loop data, no 0xFE). Re-importing all exported songs gives 83,627 bytes in total (originals: 83,902) with identical events.
- Size limit: the longest original song (9,834 bytes), because the game reserves a buffer of that size in its audio heap (0x413DC bytes) at start-up. Also checked when building.
- Imported songs are stored in `files/music/song_NN.seq` (game format), so restore points keep them.

### Which song plays where (Step 50, 2026-10-28)

- **The game:** every place has a music number (enum Music of snd.h, 0–30: title, menus, test screen, per vehicle flight / good landing / missed landing / crash, Cannonball hit/miss, Jumble Hopper goal, Birdman landing/crash, results, congratulations, ending), and the place plays the song of that number. sndSetMusic only stores the number (gCurrentMusicId, 0x803505B0); sndSetMusicState starts it with uvaSeqNew(gCurrentMusicId) (play: ROM 0xC6F04, resume after the pause menu: 0xC6F78). The game compares gCurrentMusicId ("is the results music playing?", game.c), so the number must stay the place's.

- **The module** (`module_seq_new`, module.c): both uvaSeqNew calls go through it (in every hack). It plays
    1. the mission's own song for the number, if the vehicle of the mission being flown (current_condition) has that number as flight / crash / good landing / missed landing (table per vehicle: HG 04/07/05/06, RB 08/0B/09/0A, GC 0C/0F/0D/0E, Cannonball 10/–/11/12, Sky Diving 13/16/14/15, Jumble Hopper 17/–/18/0A (its hard end jingle is the Rocket Belt's), Birdman 19/1B/1A/1F); mission entry +0x9C u8[4], song + 1;
    2. otherwise the place's song from the lower data area +0x48 u8[32] (song + 1, 0 = its own);
    3. otherwise the number itself. 0x1F is the module's own place for the Birdman's landing beside the target (and its hard end jingle "landing missed"): the game has none (it plays the crash jingle 0x1B), the module plays 0x1F there and maps it to the mission's song or to the song of place 0x1B.
- **Editor:** Music tab "Which song plays where" (place → song, `ProjectSettings.MusicPlaces`), Missions tab section "Music" (`MissionSettings.Music`, kind → song; the Birdman's landings only with points). A warning when the kind of song does not fit the place: a song without a loop where the original loops (silence afterwards), a looping song for a jingle (plays on until the next music). The import reads both tables back.
- **Checked:** emulator (RAM: which song lies in the sequence buffer, gCurrentMusicId): test screen with place 03 → song 30: song 30 playing, gCurrentMusicId 3; HG A1 with its own flight song 12: song 12 playing, gCurrentMusicId 4. Not checked: the jingles in the game, the Birdman's 0x1F, resuming after the pause menu.